

Exercice 1 :

Exercice préparatoire 1 : Déclaration et affichage des éléments d'un tableau

1. Écrivez un programme C qui :
 - Déclare un tableau de 3 entiers.
 - Demande à l'utilisateur d'entrer 3 entiers.
 - Affiche les éléments du tableau un par un.

Exemple d'entrée/sortie :

Entrée : 7 10 13

Sortie : Les éléments sont : 7 10 13

```
#include <stdio.h>

int main() {
    int arr[3];

    printf("Entrez 3 entiers :\n");
    for (int i = 0; i < 3; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Les éléments du tableau sont :\n");
    for (int i = 0; i < 3; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}
```

Exercice préparatoire 2 : Parcourir un tableau avec des pointeurs

1. Écrivez un programme C qui :
 - Déclare un tableau de 3 entiers.
 - Initialise les 3 entiers avec des valeurs fixes.
 - Affiche les éléments impairs du tableau en utilisant un pointeur.

Exemple d'entrée/sortie :

Valeurs dans le tableau : 5 8 11
Sortie : Les éléments impairs sont : 5 11

```
#include <stdio.h>

int main() {
    int arr[3] = {5, 8, 11};
    int *ptr = arr;

    printf("Les éléments impairs du tableau sont :\n");
    for (int i = 0; i < 3; i++) {
        if (*(ptr + i) % 2 != 0) {
            printf("%d ", *(ptr + i));
        }
    }
    printf("\n");

    return 0;
}
```

Exercice préparatoire 3 : Calcul de la somme des éléments d'un tableau

1. Écrivez un programme C qui :
 - Déclare un tableau de 3 entiers.
 - Demande à l'utilisateur de saisir les valeurs du tableau.
 - Calcule et affiche la somme de tous les éléments **impairs** du tableau.

Exemple d'entrée/sortie :

Entrée : 3 6 9

Sortie : La somme des éléments impairs est : 12

```
#include <stdio.h>

int main() {
    int arr[3], sum_odd = 0;

    printf("Entrez 3 entiers :\n");
    for (int i = 0; i < 3; i++) {
        scanf("%d", &arr[i]);
    }

    for (int i = 0; i < 3; i++) {
        if (arr[i] % 2 != 0) {
            sum_odd += arr[i];
        }
    }

    printf("La somme des éléments impairs est : %d\n", sum_odd);

    return 0;
}
```

Exercice préparatoire 4 : Trouver les éléments impairs

1. Écrivez un programme C qui :
 - Déclare un tableau de 3 entiers.
 - Demande à l'utilisateur d'entrer 3 entiers.
 - Affiche uniquement les éléments impairs du tableau.

Exemple d'entrée/sortie :

Entrée : 2 5 8

Sortie : Les éléments impairs sont : 5

```
#include <stdio.h>

int main() {
    int arr[3];

    printf("Entrez 3 entiers :\n");
    for (int i = 0; i < 3; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Les éléments impairs sont :\n");
    for (int i = 0; i < 3; i++) {
        if (arr[i] % 2 != 0) {
            printf("%d ", arr[i]);
        }
    }
    printf("\n");

    return 0;
}
```

Exercice préparatoire 5 : Combinaison de pointeurs et calculs

1. Écrivez un programme C qui :
 - Déclare un tableau de 3 entiers.
 - Utilise un pointeur pour :
 - Parcourir le tableau.
 - Calculer et afficher la somme des éléments **impairs**.

Exemple d'entrée/sortie :

Entrée : 4 7 9

Sortie : La somme des éléments impairs est : 16

```
#include <stdio.h>

int main() {
    int arr[3], *ptr, sum_odd = 0;

    printf("Entrez 3 entiers :\n");
    for (int i = 0; i < 3; i++) {
        scanf("%d", &arr[i]);
    }

    ptr = arr; // Initialisation du pointeur au début du tableau
    for (int i = 0; i < 3; i++) {
        if (*(ptr + i) % 2 != 0) {
            sum_odd += *(ptr + i);
        }
    }

    printf("La somme des éléments impairs est : %d\n", sum_odd);

    return 0;
}
```

Ces exercices permettent de travailler de manière progressive avec :

1. La manipulation de tableaux de 3 éléments.
2. L'utilisation de pointeurs pour parcourir les tableaux.
3. Le filtrage et le calcul sur les **éléments impairs**.

Exercice 2 :

Exercice préparatoire 1 : Affichage selon une condition

1. Écrivez un programme C qui :
 - Demande à l'utilisateur de saisir un nombre entier.
 - Affiche "Nombre positif" si le nombre est supérieur ou égal à 0.
 - Sinon, affiche "Nombre négatif".

Exemple d'entrée/sortie :

Entrée : -5
Sortie : Nombre négatif

Entrée : 10
Sortie : Nombre positif

```
#include <stdio.h>

int main() {
    int nombre;

    printf("Entrez un nombre entier : ");
    scanf("%d", &nombre);

    if (nombre >= 0) {
        printf("Nombre positif\n");
    } else {
        printf("Nombre négatif\n");
    }

    return 0;
}
```

Exercice préparatoire 2 : Choix parmi deux options

1. Écrivez un programme C qui :
 - Demande à l'utilisateur de choisir entre deux options (1 pour Café, 2 pour Thé).
 - Affiche "Vous avez choisi Café" ou "Vous avez choisi Thé" selon l'entrée.
 - Si l'entrée est invalide, affiche "Choix invalide".

Exemple d'entrée/sortie :

Entrée : 1
Sortie : Vous avez choisi Café

Entrée : 3
Sortie : Choix invalide

```
#include <stdio.h>

int main() {
    int choix;

    printf("Choisissez une option (1 pour Café, 2 pour Thé) : ");
    scanf("%d", &choix);

    if (choix == 1) {
        printf("Vous avez choisi Café\n");
    } else if (choix == 2) {
        printf("Vous avez choisi Thé\n");
    } else {
        printf("Choix invalide\n");
    }

    return 0;
}
```

Exercice préparatoire 3 : Calcul selon une condition

1. Écrivez un programme C qui :
 - Demande à l'utilisateur de saisir une valeur.
 - Si la valeur est supérieure à 100, calcule et affiche 10 % de la valeur.
 - Sinon, affiche simplement la valeur.

Exemple d'entrée/sortie :

Entrée : 150
Sortie : 10 % de 150 est : 15

Entrée : 80
Sortie : La valeur est : 80

```
#include <stdio.h>

int main() {
    float valeur, pourcentage;

    printf("Entrez une valeur : ");
    scanf("%f", &valeur);

    if (valeur > 100) {
        pourcentage = valeur * 0.10;
        printf("10 %% de %.2f est : %.2f\n", valeur, pourcentage);
    } else {
        printf("La valeur est : %.2f\n", valeur);
    }

    return 0;
}
```

Exercice préparatoire 4 : Réductions simples

1. Écrivez un programme C qui :
 - Demande à l'utilisateur de saisir un prix et une quantité.
 - Si la quantité dépasse 10, applique une réduction de 20 % au prix total.
 - Sinon, affiche le prix total sans réduction.

Exemple d'entrée/sortie :

Entrée : Prix = 100, Quantité = 12
Sortie : Prix total après réduction : 960

Entrée : Prix = 100, Quantité = 8
Sortie : Prix total : 800

```
#include <stdio.h>

int main() {
    float prix, total;
    int quantite;

    printf("Entrez le prix d'un article : ");
    scanf("%f", &prix);
    printf("Entrez la quantité : ");
    scanf("%d", &quantite);

    total = prix * quantite;

    if (quantite > 10) {
        total *= 0.80; // Réduction de 20 %
        printf("Prix total après réduction : %.2f\n", total);
    } else {
        printf("Prix total : %.2f\n", total);
    }

    return 0;
}
```

Exercice préparatoire 5 : Simuler un abonnement

1. Écrivez un programme C qui :
 - Demande à l'utilisateur de choisir un type de service (1 pour Standard, 2 pour Premium).
 - Demande ensuite une durée en mois.
 - Calcule et affiche le coût total en fonction de ces règles :
 - **Standard** : 100 DH/mois.
 - **Premium** : 200 DH/mois.

Exemple d'entrée/sortie :

Entrée : Type = 1, Durée = 10
Sortie : Le coût total est : 1000 DH

Entrée : Type = 2, Durée = 5
Sortie : Le coût total est : 1000 DH

```
#include <stdio.h>

int main() {
    int type, duree;
    float cout;

    printf("Choisissez le type de service (1 pour Standard, 2 pour Premium)
: ");
    scanf("%d", &type);
    printf("Entrez la durée (en mois) : ");
    scanf("%d", &duree);

    if (type == 1) {
        cout = 100 * duree; // Standard : 100 DH/mois
        printf("Le coût total est : %.2f DH\n", cout);
    } else if (type == 2) {
        cout = 200 * duree; // Premium : 200 DH/mois
        printf("Le coût total est : %.2f DH\n", cout);
    } else {
        printf("Type de service invalide\n");
    }

    return 0;
}
```

Ces exercices couvrent :

1. **Les bases des conditions.**
2. **La prise de décisions basées sur les entrées.**
3. **Les calculs conditionnels simples et complexes.**

Exercice 3 :

Préparation de Exercice 3 :

Exercice préparatoire 1 : Remplir et afficher une matrice

1. Écrivez un programme C qui :
 - Demande à l'utilisateur de remplir une matrice 2×2 (entiers).
 - Affiche les éléments de la matrice sous forme de tableau.

Exemple d'entrée/sortie :

Entrée :

1 2
3 4

Sortie :

1 2 3 4

```
#include <stdio.h>

int main() {
    int mat[2][2];

    printf("Entrez les éléments de la matrice 2x2 :\n");
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            scanf("%d", &mat[i][j]);
        }
    }

    printf("Les éléments de la matrice sont :\n");
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            printf("%d ", mat[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Exercice préparatoire 2 : Calculer la somme des éléments d'une matrice

1. Écrivez un programme C qui :
 - Demande à l'utilisateur de remplir une matrice 2×2 (entiers).
 - Calcule et affiche la somme de tous les éléments de la matrice.

Exemple d'entrée/sortie :

Entrée :

```
1 2
3 4
```

Sortie : La somme des éléments est : 10

```
#include <stdio.h>
```

```
int main() {
    int mat[2][2], sum = 0;

    printf("Entrez les éléments de la matrice 2x2 :\n");
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            scanf("%d", &mat[i][j]);
            sum += mat[i][j];
        }
    }

    printf("La somme des éléments est : %d\n", sum);

    return 0;
}
```

Exercice préparatoire 3 : Calculer la somme des éléments de chaque ligne

1. Écrivez un programme C qui :
 - Demande à l'utilisateur de remplir une matrice 2×2 (entiers).
 - Calcule et affiche la somme des éléments pour chaque ligne.

Exemple d'entrée/sortie :

Entrée :

1 2
3 4

Sortie :

Somme de la ligne 1 : 3
Somme de la ligne 2 : 7

```
#include <stdio.h>
```

```
int main() {  
    int mat[2][2], sum_rows[2];  
  
    printf("Entrez les éléments de la matrice 2x2 :\n");  
    for (int i = 0; i < 2; i++) {  
        sum_rows[i]=0;  
        for (int j = 0; j < 2; j++) {  
            scanf("%d", &mat[i][j]);  
            sum_rows[i] += mat[i][j];  
        }  
    }  
  
    for (int i = 0; i < 2; i++) {  
        printf("Somme de la ligne %d : %d\n", i + 1, sum_rows[i]);  
    }  
  
    return 0;  
}
```

Exercice préparatoire 4 : Trouver le plus grand élément d'une matrice

1. Écrivez un programme C qui :
 - Demande à l'utilisateur de remplir une matrice 2×2 (entiers).
 - Trouve et affiche le plus grand élément de la matrice.

Exemple d'entrée/sortie :

Entrée :

5 8
3 7

Sortie : Le plus grand élément est : 8

```
#include <stdio.h>

int main() {
    int mat[2][2] ; int max ;

    printf("Entrez les éléments de la matrice 2x2 :\n");
    for (int i = 0; i < 2; i++)
    {
        for (int j = 0; j < 2; j++)
            scanf("%d", &mat[i][j]);
    }

    max = mat[0][0];
    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 2; j++) {
            if (mat[i][j] > max) {
                max = mat[i][j];
            }
        }

    printf("Le plus grand élément est : %d\n", max);

    return 0;
}
```

Exercice préparatoire 5 : Combinaison des calculs

1. Écrivez un programme C qui :
 - Demande à l'utilisateur de remplir une matrice 3×3 (entiers).
 - Calcule et affiche la somme des éléments pour chaque ligne.
 - Trouve et affiche le plus grand élément de la matrice.

Exemple d'entrée/sortie :

Entrée :

```
1 2 3
4 5 6
7 8 9
```

Sortie :

```
Somme de la ligne 1 : 6
Somme de la ligne 2 : 15
Somme de la ligne 3 : 24
Le plus grand élément est : 9
```

```
#include <stdio.h>
```

```
int main() {
    int mat[3][3], sum_rows[3], max ;

    printf("Entrez les éléments de la matrice 3x3 :\n");

    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            scanf("%d", &mat[i][j]);
```

```
    max= mat[0][0];
    for (int i = 0; i < 3; i++) {
```

```

        for (int j = 0; j < 3; j++) {

            sum_rows[i] += mat[i][j];
            if (mat[i][j] > max) {
                max = mat[i][j];
            }
        }
    }

    printf("Somme des lignes :\n");
    for (int i = 0; i < 3; i++) {
        printf("Ligne %d : %d\n", i + 1, sum_rows[i]);
    }

    printf("Le plus grand élément est : %d\n", max);

    return 0;
}

```

Ces exercices couvrent :

1. **Les bases de la manipulation des matrices** (lecture, écriture et affichage).
2. **Les calculs simples sur des matrices** (somme, maximum).
3. **La logique nécessaire pour résoudre l'Exercice 3 de l'examen.**

Exercice 4 :

Préparation a l'exercice 4 :

Exercice préparatoire 1 : Définir et utiliser une fonction simple

1. Écrivez un programme C qui :
 - Crée une fonction `int carre(int n)` qui retourne le carré d'un nombre.
 - Demande à l'utilisateur d'entrer un nombre.
 - Appelle la fonction pour calculer le carré et affiche le résultat.

Exemple d'entrée/sortie :

Entrée : 5
Sortie : Le carré de 5 est : 25

```
#include <stdio.h>

// Fonction pour calculer le carré d'un nombre
int carre(int n) {
    return n * n;
}

int main() {
    int nombre;

    printf("Entrez un nombre : ");
    scanf("%d", &nombre);

    printf("Le carré de %d est : %d\n", nombre, carre(nombre));

    return 0;
}
```

Exercice préparatoire 2 : Fonction avec plusieurs paramètres

1. Écrivez un programme C qui :
 - Crée une fonction `int produit(int a, int b)` qui retourne le produit de deux nombres.
 - Demande à l'utilisateur d'entrer deux nombres.
 - Appelle la fonction pour calculer le produit et affiche le résultat.

Exemple d'entrée/sortie :

Entrée : 4 6
Sortie : Le produit de 4 et 6 est : 24

```
#include <stdio.h>

// Fonction pour calculer le produit de deux nombres
int produit(int a, int b) {
    return a * b;
}

int main() {
    int nombre1, nombre2;

    printf("Entrez deux nombres : ");
    scanf("%d %d", &nombre1, &nombre2);

    printf("Le produit de %d et %d est : %d\n", nombre1, nombre2,
    produit(nombre1, nombre2));

    return 0;
}

#include <stdio.h>
```

```
// Fonction pour calculer le produit de deux nombres
int produit(int a, int b) {
    return a * b;
}

int main() {
    int nombre1, nombre2;

    printf("Entrez deux nombres : ");
    scanf("%d %d", &nombre1, &nombre2);

    printf("Le produit de %d et %d est : %d\n", nombre1, nombre2,
    produit(nombre1, nombre2));

    return 0;
}
```

Exercice préparatoire 3 : Fonction avec une condition

1. Écrivez un programme C qui :
 - Crée une fonction `float reduction(float prix)` qui applique une réduction de 10 % si le prix est supérieur ou égal à 100.
 - Demande à l'utilisateur d'entrer un prix.
 - Appelle la fonction et affiche le prix final.

Exemple d'entrée/sortie :

Entrée : 150
 Sortie : Le prix final après réduction est : 135.00

```
#include <stdio.h>

// Fonction pour appliquer une réduction de 10 % si le prix est supérieur
ou égal à 100
float reduction(float prix) {
    if (prix >= 100) {
        return prix * 0.90; // Réduction de 10 %
    }
    return prix;
}

int main() {
    float prix;

    printf("Entrez un prix : ");
    scanf("%f", &prix);
```

```
printf("Le prix final après réduction est : %.2f\n", reduction(prix));  
return 0;  
}
```

Exercice préparatoire 4 : Fonction pour calculer un coût total

1. Écrivez un programme C qui :
 - Crée une fonction `float calcul_total(float prix_unitaire, int quantite)` qui retourne le coût total ($\text{prix unitaire} \times \text{quantité}$).
 - Demande à l'utilisateur d'entrer le prix unitaire et la quantité.
 - Appelle la fonction et affiche le coût total.

Exemple d'entrée/sortie :

Entrée : Prix unitaire = 20, Quantité = 5
Sortie : Le coût total est : 100.00

```
#include <stdio.h>  
  
// Fonction pour calculer le coût total  
float calcul_total(float prix_unitaire, int quantite) {  
    return prix_unitaire * quantite;  
}  
  
int main() {  
    float prix_unitaire;  
    int quantite;  
  
    printf("Entrez le prix unitaire : ");  
    scanf("%f", &prix_unitaire);  
    printf("Entrez la quantité : ");  
    scanf("%d", &quantite);  
  
    printf("Le coût total est : %.2f\n", calcul_total(prix_unitaire,  
quantite));  
  
    return 0;  
}
```

Exercice préparatoire 5 : Fonction avec réduction conditionnelle

1. Écrivez un programme C qui :
 - Crée une fonction `float calcul_prix(float prix_unitaire, int quantite)` qui :
 - Retourne le coût total avec une réduction de 10 % si la quantité est supérieure ou égale à 10.
 - Demande à l'utilisateur d'entrer le prix unitaire et la quantité.
 - Appelle la fonction et affiche le coût total.

Exemple d'entrée/sortie :

Entrée : Prix unitaire = 50, Quantité = 12
Sortie : Le coût total est : 540.00

```
#include <stdio.h>

// Fonction pour calculer le coût total avec une réduction conditionnelle
float calcul_prix(float prix_unitaire, int quantite) {
    if (quantite >= 10) {
        return prix_unitaire * quantite * 0.90; // Réduction de 10 %
    }
    return prix_unitaire * quantite;
}

int main() {
    float prix_unitaire;
    int quantite;

    printf("Entrez le prix unitaire : ");
    scanf("%f", &prix_unitaire);
    printf("Entrez la quantité : ");
    scanf("%d", &quantite);

    printf("Le coût total est : %.2f\n", calcul_prix(prix_unitaire,
quantite));

    return 0;
}
```

}

Ces exercices couvrent :

1. **La création et l'appel de fonctions simples.**
2. **Le passage de paramètres et le retour de valeurs.**
3. **Les calculs conditionnels à l'intérieur des fonctions.**
4. **Les cas d'utilisation de fonctions pour des applications réelles.**

Exercice 5

Préparation de l'exercice 5 :

Exercice préparatoire 1 : Définir et afficher une structure

1. Écrivez un programme C qui :
 - Crée une structure `Etudiant` avec les champs `nom` (chaîne de caractères), `age` (entier), et `moyenne` (flottant).
 - Déclare une variable de type `Etudiant`.
 - Demande à l'utilisateur de remplir les informations d'un étudiant.
 - Affiche les informations de l'étudiant.

Exemple d'entrée/sortie :

Entrée : Nom = Ali, Age = 20, Moyenne = 14.5
Sortie : Nom : Ali, Age : 20, Moyenne : 14.5

```
#include <stdio.h>

// Définition de la structure Etudiant
typedef struct {
    char nom[50];
    int age;
    float moyenne;
} Etudiant;

int main() {
```

```

    Etudiant etudiant;

    // Saisie des informations
    printf("Entrez le nom de l'étudiant : ");
    scanf("%s", etudiant.nom);
    printf("Entrez l'âge de l'étudiant : ");
    scanf("%d", &etudiant.age);
    printf("Entrez la moyenne de l'étudiant : ");
    scanf("%f", &etudiant.moyenne);

    // Affichage des informations
    printf("\nNom : %s, Age : %d, Moyenne : %.2f\n", etudiant.nom,
etudiant.age, etudiant.moyenne);

    return 0;
}

```

Exercice préparatoire 2 : Tableaux de structures

1. Écrivez un programme C qui :
 - Crée une structure `Livre` avec les champs `titre` (chaîne de caractères), `auteur` (chaîne de caractères), et `annee` (entier).
 - Déclare un tableau de 3 livres.
 - Demande à l'utilisateur de remplir les informations des 3 livres.
 - Affiche toutes les informations des livres.

Exemple d'entrée/sortie :

Entrée :

```

Livres 1 : Titre = C Programming, Auteur = Kernighan, Annee = 1978
Livres 2 : Titre = The Art of Computer, Auteur = Knuth, Annee = 1968
Livres 3 : Titre = Clean Code, Auteur = Martin, Annee = 2008

```

Sortie :

```

Livres 1 : Titre : C Programming, Auteur : Kernighan, Annee : 1978
Livres 2 : Titre : The Art of Computer, Auteur : Knuth, Annee : 1968
Livres 3 : Titre : Clean Code, Auteur : Martin, Annee : 2008

```

```

#include <stdio.h>

// Définition de la structure Livre
typedef struct {
    char titre[100];
    char auteur[50];
    int annee;
} Livre;

```

```

int main() {
    Livre livres[3];

    // Saisie des informations des livres
    for (int i = 0; i < 3; i++) {
        printf("Livre %d :\n", i + 1);
        printf("Titre : ");
        scanf("%s", livres[i].titre);
        printf("Auteur : ");
        scanf("%s", livres[i].auteur);
        printf("Annee : ");
        scanf("%d", &livres[i].annee);
    }

    // Affichage des informations des livres
    printf("\nInformations des livres :\n");
    for (int i = 0; i < 3; i++) {
        printf("Livre %d : Titre : %s, Auteur : %s, Annee : %d\n", i + 1,
            livres[i].titre, livres[i].auteur, livres[i].annee);
    }

    return 0;
}

```

Exercice préparatoire 3 : Rechercher un élément selon une condition

1. Écrivez un programme C qui :
 - Crée une structure `Produit` avec les champs `nom` (chaîne de caractères), `prix` (flottant), et `quantite` (entier).
 - Déclare un tableau de 3 produits.
 - Demande à l'utilisateur de remplir les informations des 3 produits.
 - Recherche et affiche le produit ayant le prix le plus élevé.

Exemple d'entrée/sortie :

Entrée :

Produit 1 : Nom = Lait, Prix = 5.5, Quantite = 10

Produit 2 : Nom = Pain, Prix = 2.0, Quantite = 20

Produit 3 : Nom = Fromage, Prix = 25.0, Quantite = 5

Sortie :

Produit le plus cher : Fromage, Prix : 25.0

```

#include <stdio.h>

// Définition de la structure Produit
typedef struct {
    char nom[50];
    float prix;
    int quantite;
}

```

```

} Produit;

int main() {
    Produit produits[3];
    int index_max = 0;

    // Saisie des informations des produits
    for (int i = 0; i < 3; i++) {
        printf("Produit %d :\n", i + 1);
        printf("Nom : ");
        scanf("%s", produits[i].nom);
        printf("Prix : ");
        scanf("%f", &produits[i].prix);
        printf("Quantite : ");
        scanf("%d", &produits[i].quantite);

        // Vérifier si le produit actuel a le prix le plus élevé
        if (produits[i].prix > produits[index_max].prix) {
            index_max = i;
        }
    }

    // Affichage du produit le plus cher
    printf("\nProduit le plus cher : %s, Prix : %.2f\n",
    produits[index_max].nom, produits[index_max].prix);

    return 0;
}

```

Exercice préparatoire 4 : Recherche dans un tableau de structures

1. Écrivez un programme C qui :
 - Crée une structure `Film` avec les champs `titre` (chaîne de caractères), `annee` (entier), et `duree` (entier, en minutes).
 - Déclare un tableau de 3 films.
 - Demande à l'utilisateur de remplir les informations des 3 films.
 - Recherche et affiche le film le plus long (ayant la durée maximale).

Exemple d'entrée/sortie :

Entrée :

Film 1 : Titre = Inception, Annee = 2010, Duree = 148

Film 2 : Titre = Titanic, Annee = 1997, Duree = 195

Film 3 : Titre = Avatar, Annee = 2009, Duree = 162

Sortie :

Le film le plus long est : Titanic, Durée : 195 minutes

```

#include <stdio.h>

// Définition de la structure Film
typedef struct {
    char titre[50];
    int annee;
    int duree; // Durée en minutes
}

```

```

} Film;

int main() {
    Film films[3];
    int index_max = 0;

    // Saisie des informations des films
    for (int i = 0; i < 3; i++) {
        printf("Film %d :\n", i + 1);
        printf("Titre : ");
        scanf("%s", films[i].titre);
        printf("Annee : ");
        scanf("%d", &films[i].annee);
        printf("Duree (en minutes) : ");
        scanf("%d", &films[i].duree);

        // Vérifier si le film actuel est le plus long
        if (films[i].duree > films[index_max].duree) {
            index_max = i;
        }
    }

    // Affichage du film le plus long
    printf("\nLe film le plus long est : %s, Durée : %d minutes\n",
        films[index_max].titre, films[index_max].duree);

    return 0;
}

```

Exercice préparatoire 5 : Combinaison des concepts

1. Écrivez un programme C qui :
 - Crée une structure `Etudiant` avec les champs `nom` (chaîne de caractères), `moyenne` (flottant), et `age` (entier).
 - Déclare un tableau de 3 étudiants.
 - Demande à l'utilisateur de remplir les informations des 3 étudiants.
 - Recherche et affiche l'étudiant ayant la meilleure moyenne.

Exemple d'entrée/sortie :

Entrée :

Etudiant 1 : Nom = Ali, Moyenne = 14.5, Age = 20
 Etudiant 2 : Nom = Sara, Moyenne = 16.2, Age = 21
 Etudiant 3 : Nom = Karim, Moyenne = 13.8, Age = 22

Sortie :

L'étudiant ayant la meilleure moyenne est : Sara, Moyenne : 16.2

```

#include <stdio.h>

// Définition de la structure Etudiant
typedef struct {
    char nom[50];
    float moyenne;
    int age;
} Etudiant;

```

```

int main() {
    Etudiant etudiants[3];
    int index_max = 0;

    // Saisie des informations des étudiants
    for (int i = 0; i < 3; i++) {
        printf("Etudiant %d :\n", i + 1);
        printf("Nom : ");
        scanf("%s", etudiants[i].nom);
        printf("Moyenne : ");
        scanf("%f", &etudiants[i].moyenne);
        printf("Age : ");
        scanf("%d", &etudiants[i].age);

        // Vérifier si l'étudiant actuel a la meilleure moyenne
        if (etudiants[i].moyenne > etudiants[index_max].moyenne) {
            index_max = i;
        }
    }

    // Affichage de l'étudiant avec la meilleure moyenne
    printf("\nL'étudiant ayant la meilleure moyenne est : %s, Moyenne : %.2f\n", etudiants[index_max].nom, etudiants[index_max].moyenne);

    return 0;
}

```

Ces exercices couvrent :

1. **La définition et l'utilisation de structures.**
2. **Les tableaux de structures et leur manipulation.**
3. **La recherche de valeurs maximales dans des données.**
4. **L'affichage formaté des résultats.**